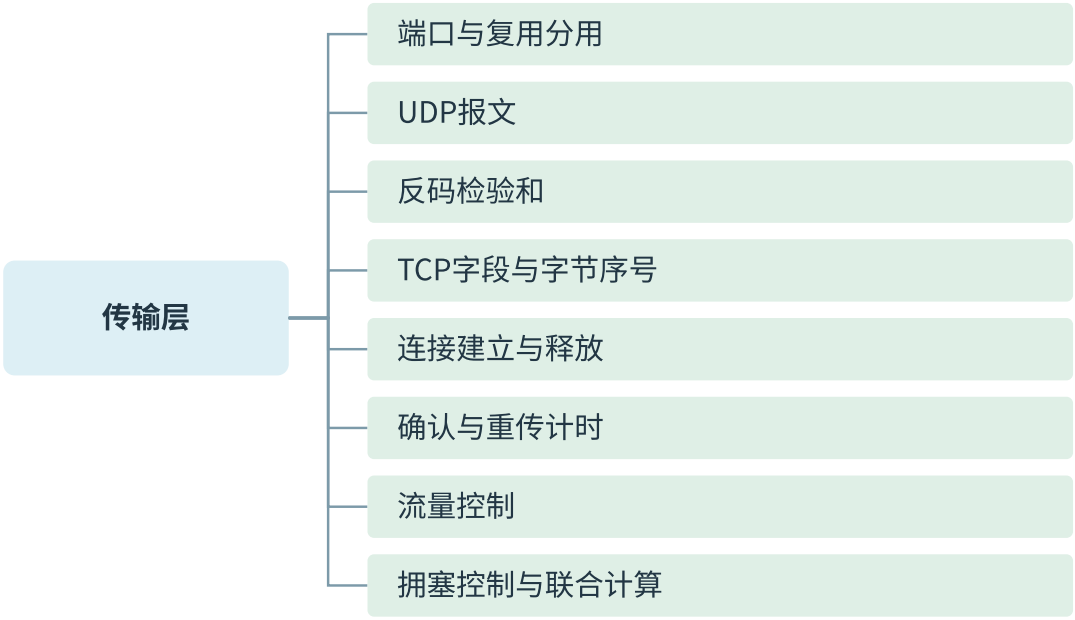
传输层 5

章节脉络



IP 把数据送向目的主机，传输层再把数据交给相应应用端点。UDP 保留报文边界但不保证可靠交付；TCP 提供有序字节流，并依靠确认、重传、窗口和连接状态来管理这条字节流。计算 TCP 时始终把字节编号、窗口额度与尚可发送量分开记录。

本章 TCP 序号按 32 位模运算；小数值例题不跨回绕点时可直接作整数加减。MSS 是一个 TCP 段可携带的最大应用数据量，不包括 TCP、IP 首部。题目写 KB 时沿用该题的教材单位，窗口与 MSS 相除时保持同一口径。

课程连接·套接字由操作系统管理

应用通过套接字接口使用传输服务，端口参与内核分用，进程号标识操作系统中的进程。一个服务器进程可接受多条连接，故不能把端口、进程和连接按一一对应关系死记。

5.1 端口与套接字：一个 IP 为什么能运行多个服务

定义 5.1.1 (传输层端点标识)

端口号是传输层区分本机应用端点的 16 位标识。熟知端口常给固定服务使用，客户端通常选临时端口。端口号不是操作系统进程号：一个服务进程可以管理多个连接，一个应用也可创建多个套接字。

传输层把多个应用的数据交给 IP，称为复用；收到后交给相应应用端点，称为分用。UDP 在通常教材模型中按目的端口等本地端点信息分用；TCP 连接由源 IP、源端口、目的 IP、目的端口共同区分。实际 UDP 套接字的绑定和连接模式还可能限定远端信息，题目未给出时按基本分用模型判断。

5.1.1 例 5-1　谁决定这个 UDP 数据报交给哪个本机服务

例 5.1.1 (配套例题)

配套例题。UDP 协议实现分用时所依据的头部字段是（　）。

A. 源端口号　B. 目的端口号　C. 长度　D. 校验和

解析。目的端口指出本机接收应用端点，选 **B**。源端口可供接收方回复时使用，长度用于界定 UDP 报文，校验和用于检错；三者都不能替代目的端口的基本分用作用。

两个 TCP 客户可以同时访问服务器的 80 端口，因为它们的源 IP 或源端口不同，四元组不同。不能因为 “服务端端口相同” 就断言只有一条连接。

不变量·UDP 长度包含 8 字节首部

应用数据 12 B 时 UDP 长度为 20 B；普通 IPv4 还要把 IP 首部计入整个数据报上限。UDP 自身保留报文边界，承载它的 IP 分片则必须先在目的 IP 层重组，不能把每个 IP 片都当一个 UDP 报文。

5.2 UDP：8 字节首部保留了哪些边界

UDP 首部只有源端口、目的端口、长度、检验和四个字段，各 16 bit，共 8 B；没有序号、确认号和窗口字段。下图每行 32 bit，两个 16 bit 字段正好占一行。

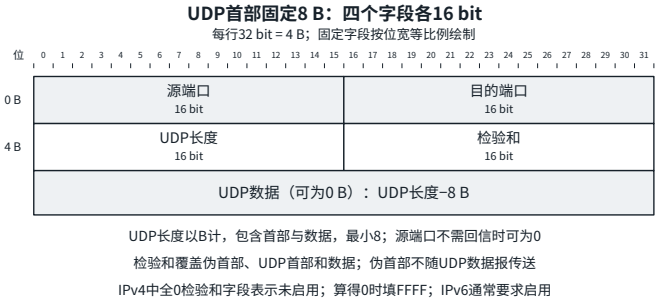


Figure 5.1: UDP 首部两行共 8 B，四个字段各 16 bit；数据从 UDP 起点之后 8 B 开始

源端口供回复时使用，不需回信时可为 0；目的端口用于基本分用。长度字段以 B 为单位，包含 UDP 首部和数据，最小 8，不能把它当成数据区长度。应用提交 12 B 数据，则 UDP 长度为 20。16 bit 长度字段最大可表示 65535 B，但普通 IPv4 还限制整个 IP 数据报不超过 65535 B；若 IPv4 首部 20 B，UDP 数据最多为 $65535-20-8=65507$ B，不能把 UDP 字段上界直接当作应用可提交的载荷上界。

UDP 直接保留应用报文边界，接收操作可区分每个 UDP 数据报；TCP 则可能把多个应用写入合并，或把一次写入拆成多个段。UDP 自身不把过长应用报文拆成多个 UDP 数据报，但 IP 可能将承载它的 IP 数据报分片；待目的主机的 IP 层重组后，才把完整 UDP 数据报交给 UDP 校验并分用。

UDP 无须三次握手，不提供自身的顺序恢复、确认重传和拥塞窗口机制。报文丢失、重复或乱序需要应用按需求处理。无连接既不表示应用完全没有状态，也不表示 UDP 的校验和可以保证可靠交付。音视频、DNS 等可以使用 UDP，但具体应用仍可能同时支持 TCP 或其他可靠机制。

UDP 开销小的直接含义是基本首部较短、没有连接建立所需往返；“一定比任何 TCP 应用快” 还取决于数据量、丢包率和应用处理。实时业务选择不等待迟到的旧报文，不等于连检错也可以省掉。

比较维度	UDP	TCP
交给应用的数据单位	一个个报文，保留边界	有序字节流，不保留应用写入边界
通信准备与端点	无须建立传输连接；底层允许时可单播、组播或广播	先建立一对端点间的全双工连接，不提供广播/组播连接
基本首部与长度	固定 8 B；有 UDP 总长度字段	20～60 B；数据偏移表示首部长，没有独立 TCP 总长度字段
丢失、次序与重复	协议本身不确认重传、不恢复次序	序号、累计确认和重传等保证可靠有序字节流
发送额度	没有自身的接收窗口和拥塞窗口机制	同时受接收窗口、拥塞窗口及在途数据限制

两者都有端口和检错机制，都借助 IP 传送。UDP 应用若需要可靠性和速率管理，这些机制必须由应用自身或其上层承担；“UDP 不提供”不能推出“使用 UDP 的应用不能提供”。TCP 有可靠传输机制，也不承诺固定时延或固定吞吐量。

错项排除·伪首部不是真的多发一层

校验借用源 IP、目的 IP、协议和长度，是为检查交付对象；它只参加计算。奇数字节补 0 也只补计算过程。因此计算线上长度和吞吐量时，不能再给 UDP 额外增加伪首部和求和补位。

5.3 反码校验和：进位为什么要加回低位

UDP 校验覆盖伪首部、UDP 首部和数据。IPv4 伪首部包含源 IP、目的 IP、0、协议号 17 和 UDP 长度，只参与计算，不作为 UDP 数据再发出。有了它，数据报即使被投递到另一地址或另一协议，其中一部分错误也能被检出；TCP 采用相应伪首部，协议号为 6，不能沿用 UDP 的 17。

按 16 位分组求二进制反码和，最高位产生的进位回卷加到最低位，反复处理直至没有进位，最后按位取反。计算前校验和字段置 0；数据总字节数为奇数时补一个 0 字节参与计算，该补齐字节不属于发送的 UDP 数据，也不计入长度字段。

5.3.1 例 5-2　少做一次回卷，会错在哪一位

例 5.3.1 (配套例题)

配套例题。若 UDP 协议在计算校验和过程中，计算得到中间结果为 `1011 1001 1011 0110` 时，还需要加上最后一个 16 位数 `0110 0101 1100 0101`，则最终计算得到的校验和是（　）。

A. `0001 1111 0111 1011`

B. `0001 1111 0111 1100`

C. `1110 0000 1000 0011`

D. `1110 0000 1000 0100`

解析。两数分别为十六进制 B9B6、65C5，普通相加得到 `1 1F7B`。保留低 16 位 1F7B，再加回最高进位 1，得到 1F7C，最后取反为 E083，即 **C**。D 对应 1F7B 直接取反，遗漏回卷；A、B 都还没完成最后取反。

接收端把收到的校验和也加入同样求和，结果应为全 1。若发送方算得的校验和为全 0，UDP 字段应写全 1；在 IPv4 中全 0 字段另表示未使用 UDP 校验。IPv6 通常要求 UDP 校验，不能把 IPv4 的可选零字段推广成 IPv6 通则。〖5.2-008〗

反码检验和与 CRC 一样，检验通过只能说明没有**检测出**差错，不能排除所有抵消或漏检情形。把三种首部中同名的检验和放在一起，必须分别核对覆盖范围与零值语义：

比较项	IPv4 首部检验和	UDP 检验和	TCP 检验和
参加计算的内容	IPv4 首部，含选项与填充	伪首部、UDP 首部、UDP 数据	伪首部、TCP 首部、TCP 数据
IPv4 伪首部	不使用	12 B：源 IP、目的 IP、0、17、UDP 长度	12 B：源 IP、目的 IP、0、6、TCP 长度
字段为全 0 时	没有关闭首部校验的含义	IPv4 表示未启用；IPv6 通常要求启用；算得 0 时传全 1	没有以全 0 关闭校验的约定

伪首部借用网络层地址来检查交付对象，只在计算时构造，不成为线上多出来的 12 B 首部。IPv4 改 TTL 只需相应更新 IP 首部检验和；NAT 若改 IP 地址，启用的 TCP/UDP 校验也可能需更新，因为伪首部变了。UDP 为奇数数据补齐的 0 字节同样只服务于求和，既不发送也不计入 UDP 长度。

最小反例·双向 seq 各有自己的坐标

甲已发 2001～4000，乙回复 seq=4001、ack=3001：乙的 seq 属于反方向，只确认甲到 3000 的是 ack。把乙的 seq 拿来确认甲的数据，会错误释放仍在途的 3001～4000。

5.4 TCP 序号、确认号与控制位怎样一起读

TCP 基本首部 20 B，含选项与填充时可到 60 B。图中沿用主书经典格式：源、目的端口各 16 bit，序号与确认号各 32 bit，数据偏移 4 bit、保留 6 bit、六个控制位各 1 bit，再接 16 bit 接收窗口；最后一行固定字段是检验和与紧急指针，各 16 bit。现代扩展可能重新赋义部分原保留位，须按题图及扩展条件解释，本图保持主书的六控制位口径。

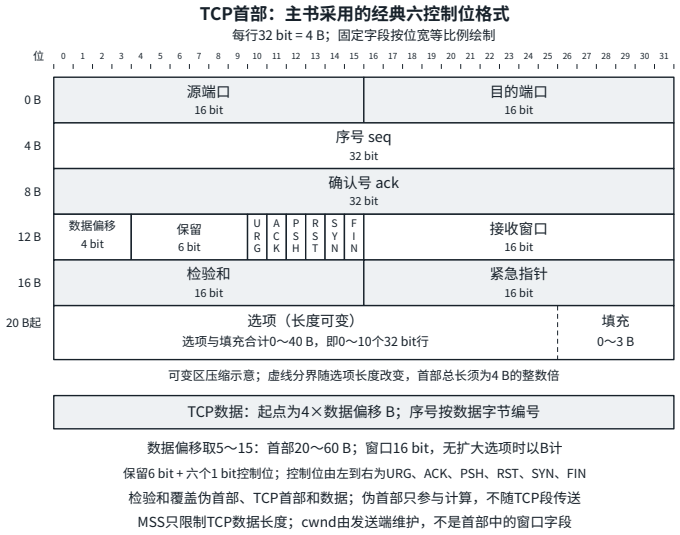


Figure 5.2: TCP 固定首部五行，每行 32 bit；数据偏移 4 B 单位，接收窗口 16 bit，选项与填充另占 0～40 B

数据偏移表示 TCP 数据起点相对 TCP 段起点的距离，以 4 B 为单位，故也就是 TCP 首部长度。取 5 时 20 B，取 15 时 60 B；加入选项后须用必要填充对齐到 4 B。它不是本段数据字节数。对未分片或已重组的普通 IPv4 分组，若其 IP 数据只承载一个 TCP 段，则

$$L_{TCP\text{数据}} = IP\text{总长度} - 4\times IHL - 4\times TCP\text{数据偏移}.$$

例如 IP 总长度 100 B、IHL=6、TCP 数据偏移 =6，TCP 数据为 52 B。不能固定减 40 B，因为两种首部都可能有选项；更不能对每个未重

组 IP 分片分别照减一个 TCP 首部。MSS 限制的是这个 TCP 数据部分，不包括任何 IP 或 TCP 首部。

控制位按条件生效：ACK=1 时确认号有效；SYN 用于连接同步，FIN 表示本方向没有更多数据；RST 用于复位或拒绝不适当的连接/报文；PSH 要求接收方及时向应用交付已收到的数据，不表示必须立即回一个 ACK。URG=1 时 16 bit 紧急指针有效，它相对本段序号指示紧急数据边界；教材把紧急数据放在数据区前部，此时紧急指针可按紧急数据的字节数理解，它不是另一套独立的 32 bit 序号。控制位语义与序号消耗要分别判断，并非任意一个控制位置 1 都“占一个序号”。

接收窗口字段是**本段发送者作为接收方**通告给对端的额度，未协商窗口扩大时以 B 计，最大 65535 B。它与本段 ack 一起描述对端可发送的字节范围；本段 seq 则描述发送者在反方向的字节位置。cwnd 由发送端根据网络反馈维护，首部中并没有名为 cwnd 的字段。下文的窗口题正是把接收窗口、拥塞窗口与在途数据这三个量放在一起计算。

TCP 按**字节**编号。数据段的 seq 是该段第一个数据字节的序号；携带 L B 时，覆盖 seq 至 $seq+L-1$，下一字节为 $seq+L$。确认号 ack 表示下一个期望按序收到的字节，因此它累计确认的是 ack 之前的连续字节，不表示“第 ack 个报文段”。收到 ack=100 表示已按序收到至 99。〖5.3-011〗

课程连接·状态是事件触发的记录

连接状态可像操作系统进程状态一样按事件画转换：发 FIN、收到其 ACK、收到对方 FIN 分别触发不同动作。仅看到报文名字不足以决定两端都已关闭，还要确定报文在何时被哪端收到。

5.4.1 例 5-3　最后收到的字节与确认号相差 1

例 5.4.1 (配套例题)

配套例题。主机甲与主机乙之间已建立一个 TCP 连接，甲向乙发送了两个连续 TCP 段，分别包含 300 B 和 500 B 有效载荷，第一个段的序列号为 200，乙正确接收到两个段后，发送给甲的确认序列号是（　）。

A. 500　B. 700　C. 800　D. 1000

解析。第一段覆盖 200～499，第二段从 500 开始，覆盖 500～999。下一个期望字节为 1000，选 **D**。不能把载荷总长 800 直接当确认号，也不能只给出最后字节 999。

SYN 和 FIN 各占一个序号，纯 ACK 不占；若 SYN 的序号为 x，则正常数据（不带 SYN）的第一个字节序号为 $x+1$。某方向 FIN 的序号等于该方向最后数据字节的下一号；对端确认这个 FIN 时还要再加 1。这一加 1 不代表增加了一个应用数据字节。

5.5 三次握手与连接释放：状态变化由哪个事件触发

三次握手同时确认两个方向的初始序号和收发可达性。客户先发 SYN（seq=x）并进入 SYN-SENT；监听服务器收到后发 SYN+ACK、seq=y、ack=$x+1$，并进入 SYN-RECEIVED；客户收到后发 ACK、seq=$x+1$、ack=$y+1$ 并进入 ESTABLISHED；服务器收到此 ACK 后也进入 ESTABLISHED。第三次 ACK 由客户发出，序号按客户方向编号，也可携带数据；不能因此把两端的初始序号当成同一个值。

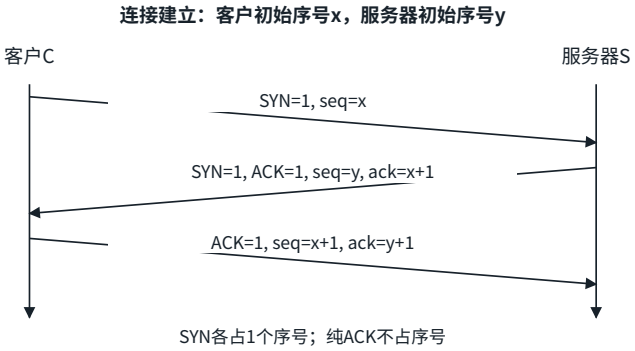


Figure 5.3: 两端各占用一个 SYN 序号；第三次确认不再额外消耗序号

错项排除·半关闭仍允许反向数据

被动端收到 FIN 后进入 CLOSE-WAIT，表示对方不会再发送，本端应用仍可继续向对方发送。把 TCP 全双工看成一根双向同时关闭的管道，会错误断言收到 FIN 后所有方向的数据都不能发。

第三次握手针对的是迟到的旧 SYN：若缺了它，服务器可能仅凭这样一个旧 SYN 就长期保留一条对端并不承认的连接。它同时确认客户已经收到服务器的初始序号。只说 “让双方知道能通信” 过于含糊，应分别指出谁收到哪个序号、确认了哪个方向。

TCP 是全双工，每个方向独立关闭。主动关闭者发送 FIN 后进入 FIN-WAIT-1；收到对自己 FIN 的 ACK 进入 FIN-WAIT-2；收到对方 FIN 并发送 ACK 后进入 TIME-WAIT。被动关闭者收到 FIN、发 ACK 后进入 CLOSE-WAIT，表示对端不再发送，但本方向应用仍可继续发送；本机应用关闭后发 FIN 进入 LAST-ACK，收到确认后才进入 CLOSED。ACK 与 FIN 可以合并，因此报文数不必总为四份，但两方向的关闭语义都必须完成。

TIME-WAIT 通常持续 2MSL，一方面在最后的 ACK 丢失时还能再次响应对方重传的 FIN，另一方面使旧连接的残留报文逐渐消失。它一般出现在完成主动关闭的一端；“每端都要固定等 2MSL 才关” 不成立。同时关闭还可能经过 CLOSING 等状态，应按收到 FIN 和 ACK 的次序判断。

5.5.1 例 5-4　至少关闭时间要从 FIN 开始画

例 5.5.1 (配套例题)

配套例题。假设客户 C 和服务器 S 已建立 TCP 连接，RTT=50 ms，最长报文段寿命 MSL=800 ms，数据传输结束后 C 主动请求断开连接。若从 C 主动向 S 发出 FIN 段时刻算起，则 C 和 S 进入 CLOSED 状态所需的时间至少分别是（　）。

A. 850 ms，50 ms　　B. 1650 ms，50 ms

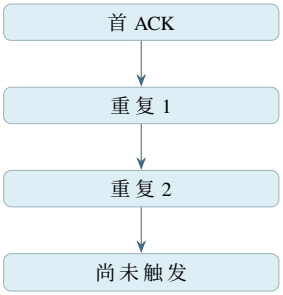
C. 850 ms，75 ms　　D. 1650 ms，75 ms

解析。求最少时间，可让 S 收到 FIN 后立即合并发送 ACK 与 FIN，忽略处理和段发送时延，单向传播为 25 ms：S 在 25 ms 收到并回复；C 在 50 ms 收到 S 的 FIN，发最后 ACK，同时开始 2MSL 计时；S 在 75 ms 收到最后 ACK 而关闭；C 在 $50 + 2 \times 800 = 1650$ ms 关闭。因此选 **D**。服务器的 CLOSED 须等最后 ACK 真正到达，不能在它发送 FIN 时就提前标记。

5.6 确认、重传与 RTT 估计

最小反例·首次加两次还差一次

先收到把确认号推进到 3001 的正常 ACK，再收到两份 ack=3001，重复计数只有 2。第三份重复 ACK 到达才满足传统快重传阈值；同号 ACK 共有 4 份中的第一份不能混算成重复。



三个同号 ACK 仅含两份重复 ACK

TCP 以累计 ACK 报告连续接收进度。若 1000～1999 丢失，而 2000～2999 先到，基本累计确认仍指向 1000；接收方可以缓存失序数据，但不能把累计 ACK 越过缺口。选择确认（SACK）选项能额外报告已收到的分块范围，但它只是补充信息，并不改变累计 ACK 的基本定义。

发送端保留未确认数据并运行重传计时器。超时重传针对未确认内容，不要求把整条连接已经确认过的所有数据重发。连续收到 3 个重复 ACK 可触发快重传，**3 个重复**中不包含最初那份正常 ACK，正是它把确认号推进到了该值；图中应逐次计数。

RTO 需要考虑 RTT 的变化。常见平滑估计为

$$SRTT \leftarrow (1-\alpha)SRTT + \alpha RTT_{\text{sample}},$$

再结合 RTT 偏差设置超时阈值。若题目明确 $RTO = \beta SRTT$，就按该给定简化式计算，不把它推广成所有实现。重传之后收到的 ACK 难以判断确认的是原段还是重传段，因此 Karn 原则不用这种有歧义的样本更新 RTT 估计；超时还会使 RTO 退避。

例如旧 SRTT 为 100 ms、$\alpha = 1/8$、新无重传样本 140 ms，则新 SRTT 为 105 ms，而不是直接改为 140 ms。若同一段曾重传，先判断样本是否可用，再代入公式；只知道 ACK 的到达时间，还不足以确定它对应哪个发送时刻。

可靠传输控制 “同一字节是否按序到齐”，拥塞控制决定 “现在允许多少字节在路上”，流量控制考虑 “接收端还有多少空间”。丢失处理会同时改变确认状态、重传状态和拥塞状态，做综合题时需分别记录。

5.7 接收窗口：总额度与剩余额度怎样区分

接收方通告的 rwnd 反映当前愿意接收的字节范围，常以累计确认号为左边界。若 ack=3001、rwnd=4000 B，则允许的接收序号区间为 3001～7000，右端的下一号为 7001。发送端已经发出但未确认的字节仍占该区间，不能把 4000 B 全部当成新数据额度。

发送窗口是字节额度，不是 “能发几个段”。窗口 2000 B、MSS1000 B，在没有已发未确认数据时可容纳 2 个满段；若已有 1000 B 在途，最多再发 1000 B。〖5.3-010〗〖5.3-029〗这类题只改动总窗口的数值，两次通告之差并不等于总窗口。

接收方通告零窗口后，发送方暂停普通数据发送，但应启动持续计时器，定时发送零窗口探测，防止 “窗口重新打开” 的通知丢失后双方永久等待。持续计时器不是重传计时器、保活计时器，也不是 TIME-WAIT 的 2MSL。〖5.3-014〗

当窗口尺度选项参与计算时，首部字段与实际字节窗口还需按协商尺度换算；选项只能在连接建立时按规则协商。题目若未给窗口扩大，按字段直接表示的字节额度处理，不自行添加尺度。

5.8 慢开始、拥塞避免与 Reno 的丢包反应

拥塞窗口 cwnd 由发送方根据网络反馈维护，慢开始门限 ssthresh 区分增长阶段。以下数值推演采用教材逐段确认、持续有数据、没有其他瓶

颈的简化模型；用 MSS 计窗口，初值按具体题目给定。图示或题源取 1 MSS 时，不把它当成现代所有 TCP 实现的固定初始窗口。

不变量·按 ACK 事件依次更新四账

先更新累计确认，再核对已发未确认区间，随后按拥塞算法改变 cwnd 并读当前 rwnd，最后取小扣在途。若只把两个窗口数字取小就回答还能发多少，会在所有有在途数据的题目中多算。

慢开始每收到新数据的 ACK 适当增加 cwnd，在一轮中所有段均分别得到确认时，窗口约每 RTT 翻倍；达到门限后转入拥塞避免，平均每 RTT 约增加 1 MSS。实际按 ACK 更新，若有延迟 ACK、应用暂时无数据或接收窗口受限，逐轮翻倍模型未必精确。

以初始 1、门限 8 为例，逐轮发送窗口可为 1、2、4、8、9、10……。若第 8 轮发送时 cwnd=12 并发生超时，按该题模型将门限设为 6，cwnd 退回 1，后续轮次为 1、2、4、6、7……；从初始第 1 轮开始数，第 13 轮为 7。〖5.3-033〗必须先把“轮次记录的是发送前还是确认后”固定，不能在超时处跳过一轮或把 12 继续保留。

超时表明没有及时得到反馈，教材 Reno 模型采用减半门限、cwnd 退回 1 再慢开始。3 个重复 ACK 触发快重传时，说明仍有后续段到达，Reno 进入快恢复：把门限减半，临时把 cwnd 设为门限加 3 MSS，用来反映已经离开网络的重复确认段，后续重复 ACK 还可继续调整；新 ACK 确认缺失数据后，窗口回到门限并进入拥塞避免。课件按传播轮次绘制的简化折线通常记录新 ACK 之后的稳定点，此时 $cwnd = ssthresh$；不要把恢复期间临时的 $+3$ 套进稳定轮次图。Tahoe 收到 3 个重复 ACK 后也退回慢开始，不能把 Reno 的恢复规则无条件套给所有版本。

在规范实现中，门限减半通常还要参考在途数据量等条件；408 教材题经常直接给出“丢包时拥塞窗口”并按其一半设门限。应使用题目明确的算法及数值条件，不以术语相同掩盖模型差异。

5.8.1 例 5-5　没有拥塞，不代表一定指数增长

例 5.8.1 (配套例题)

配套例题。若主机甲与主机乙已建立 TCP 连接，MSS 为 1 KB，RTT 为 2 ms，则在不出现拥塞的前提下，拥塞窗口从 8 KB 增长到 32 KB 所需的最长时间是（　）。

A. 4 ms　　B. 8 ms　　C. 24 ms　　D. 48 ms

解析。“最长”需要考虑门限可能已不大于 8 KB，从起点便处于拥塞避免；每 RTT 增加 1 KB，从 8 到 32 需 24 个 RTT，共 48 ms，选 **D**。若始终处于慢开始，8→16→32 只需 2RTT=4 ms，是较短情形。原题沿用持续有数据、ACK 正常返回的教材增长模型；不能把接收端的任意停顿也算进这个“最长”里。

5.9 两个窗口与在途数据：本次究竟还能发多少

教材常用总发送额度

$$W = \min(cwnd, rwnd).$$

若最新有效确认与窗口左沿一致，已发未确认字节数为 F，那么可新发额度为

$$A = \max(0, W - F).$$

这个式子要求先确定 ACK 确认了哪些字节、cwnd 如何因该 ACK 变化。若题图给出了右边界或不同时间的窗口更新，应直接画出序号区

间，不能拿过期 rwnd 替代当前通告。某个段何时收到 ACK、另一个段是否仍在途，这些中间状态才是窗口题的关键。

比较量	谁维护，依据什么	回答哪个问题
接收窗口 rwnd	接收方按可用接收能力通告，来自 TCP 窗口字段及协商尺度	对端愿意接收多大字节范围
拥塞窗口 cwnd	发送方按确认、丢包等反馈维护，不是首部通告字段	网络反馈允许多少数据在途
已发未确认量 F	发送方按已发送区间与累计确认进度登记	总额度中已经占用了多少
可新发额度 A	以当前两个上限取小，再扣除 F，下限为 0	本时刻还能新发多少字节

不变量·按 ACK 事件依次更新四账

先更新累计确认，再核对已发未确认区间，随后按拥塞算法改变 cwnd 并读当前 rwnd，最后取小扣在途。若只把两个窗口数字取小就回答还能发多少，会在所有有在途数据的题目中多算。

接收窗口缩小不直接证明网络拥塞，cwnd 降低也不证明接收缓存已满。快重传、超时、零窗口探测分别触发不同动作；可靠性、拥塞与流量控制可以相互作用，但不能用一个“窗口变小”替代全部状态判断。

一次 ACK 事件的四本账

第一本记录已按序确认到哪里：由新 ack 减去旧左沿确定释放的字节。第二本记录还有多少数据在途，不能把反方向 seq 当作本方向确认。第三本按题设慢开始、拥塞避免、超时或重复 ACK 事件更新 cwnd。第四本读取当前有效 rwnd，取两上限之小后再扣在途量，才得到还能新发的字节。

可新发额度公式来自集合差：当前总窗口允许的字节区间中，已经发出尚未确认的部分不可再算作新数据；扣除其长度 F 以后才能安排新发送。若总额度小于已在途量，新增额度按 0 处理，而不是要求撤回已进入链路的数据。这个推导默认最新窗口左沿与累计 ACK 一致，复杂窗口更新以实际序号区间为准。

5.9.1 例 5-6　ACK 只移走已经确认的那一段

例 5.9.1 (配套例题)

配套例题。主机甲通过 TCP 向主机乙发送数据的部分过程如下图。seq 为序号，ack_seq 为确认序号，rcvwnd 为接收窗口。甲在 t_0 的拥塞窗口和发送窗口均为 2000 B，拥塞控制阈值为 8000 B，MSS=1000 B，甲始终以 MSS 发送 TCP 段。若甲在 t_1 收到图示确认段，则未收到新的确认之前，还可以继续发送的 TCP 段数是（　）。



Figure 5.4: 原题数据：第二个数据段仍未被 ack_seq=3001 确认

A. 2　B. 3　C. 4　D. 5

解析。第一段 seq=2001 覆盖 2001～3000，第二段 seq=3001 覆盖 3001～4000。返回 ack_seq=3001 只累计确认第一段，第二段的 1000 B 仍在途。乙段的 seq=4001 属于乙 → 甲方向，不能拿它确认甲发出的字节。

甲原 cwnd 为 2000 B，小于门限 8000 B，收到一份对新数据的确认后，按题设慢开始增加 1 MSS，cwnd 变为 3000 B；rwnd 通告为 4000 B，总发送额度取 3000 B。减去在途 1000 B，尚可新发 2000 B，即 2 段，选 **A**。对应允许的甲方发送范围从 3001 起共 3000 B，其中 3001～4000 已经发出，4001～6000 还可新发。

不变量·按 ACK 事件依次更新四账

先更新累计确认，再核对已发未确认区间，随后按拥塞算法改变 cwnd 并读当前 rwnd，最后取小扣在途。若只把两个窗口数字取小就回答还能发多少，会在所有有在途数据的题目中多算。

5.9.2 例 5-7　拥塞窗口变大后，接收窗口仍可成为上限

例 5.9.2 (配套例题)

配套例题。主机甲和乙已建立 TCP 连接，甲始终以 MSS=1 KB 大小的段发送数据，并一直有数据发送；乙每收到一个数据段都会发出一个接收窗口为 10 KB 的确认段。若甲在 t 时刻发生超时时拥塞窗口为 8 KB，则从 t 起不再发生超时的情况下，经过 10 个 RTT 后，甲的发送窗口是（　）。

A. 10 KB　B. 12 KB　C. 14 KB　D. 15 KB

解析。超时后门限为 4 KB，cwnd 退回 1 KB。按本题逐轮教材模型，前 8 个 RTT 后的 cwnd 依次为 2、4、5、6、7、8、9、10 KB，发送额度到达接收方上限 10 KB。之后没有新的丢包，cwnd 不会因超时下降，而固定 rwnd 仍把发送窗口限制在 10 KB，因此第 10 个 RTT 后仍为 10 KB，选 **A**。达到接收方上限后，无须继续假定 cwnd 每轮恰好增长 1 KB 来选答案；窗口受限时的实际 ACK 增长轨迹也未必遵循未受限时的逐轮简化式。

上题 rwnd 固定，故无须再算接收缓存的占用；若题目改问 “还可发送多少”，还必须再减在途数据，不能直接回答 10 KB。窗口题可按 “序号区间—ACK 覆盖—cwnd 事件—rwnd 边界—扣除在途” 的顺序推进。

5.9.3 例 5-8　从完整字节表同时读出握手、填充与已确认数据

例 5.9.3 (配套例题)

配套例题（改编）。　主机 H 通过快速以太网连接 Internet，IP 地址为 192.168.0.8；服务器 S 的 IP 地址为 211.68.71.80。H 与 S 使用 TCP 通信，在 H 上捕获的 5 个 IP 分组及其中一个分组在 S 发出时的前 40 字节如下图，均为十六进制、按网络字节序排列。**第 2 包首字节采用 45，第 3 包 TCP 数据偏移与控制字节采用 50 10**。

2012改编：在H捕获的5个IP分组及S发出的对应分组

每项仅列前40 B；每行20 B，均为网络字节序的十六进制

1	45 00 00 30 01 9b 40 00 80 06 1d e8 c0 a8 00 08 d3 44 47 50 0b d9 13 88 84 6b 41 c5 00 00 00 00 70 02 43 80 5d b0 00 00
2	45 00 00 30 00 00 40 00 31 06 6e 83 d3 44 47 50 c0 a8 00 08 13 88 0b d9 e0 59 9f ef 84 6b 41 c6 70 12 16 d0 37 e1 00 00
3	45 00 00 28 01 9c 40 00 80 06 1d ef c0 a8 00 08 d3 44 47 50 0b d9 13 88 84 6b 41 c6 e0 59 9f f0 50 10 43 80 2b 32 00 00
4	45 00 00 38 01 9d 40 00 80 06 1d de c0 a8 00 08 d3 44 47 50 0b d9 13 88 84 6b 41 c6 e0 59 9f f0 50 18 43 80 e6 55 00 00
5	45 00 00 28 68 11 40 00 31 06 06 7a d3 44 47 50 c0 a8 00 08 13 88 0b d9 e0 59 9f f0 84 6b 41 d6 50 10 16 d0 57 d2 00 00
S	45 00 00 28 68 11 40 00 40 06 ec ad d3 44 47 50 ca 76 01 06 13 88 a1 08 e0 59 9f f0 84 6b 41 d6 50 10 16 d0 b7 d6 00 00

第2包首字节45；第3包TCP控制字节50 10；其余字节按主书表保留

Figure 5.5: 1～5 为 H 处捕获的分组，S 行为在服务器发出时捕获的对应分组；每行 20 B

字段布局参照第 4 章 IPv4 首部格式和本章TCP 首部图：IPv4 源、目的地址分别位于第 13～16、第 17～20 字节，TTL 位于第 9 字节；IHL 与 TCP 数据偏移字段均以 4 B 为单位。

（1）5 个 IP 分组中，哪几个由 H 发送？哪几个完成 TCP 连接建立过程？哪几个在通过快速以太网传输时进行了填充？

（2）根据这 5 个 IP 分组，分析 S 已经收到的应用层数据有多少字节。

（3）若某个 IP 分组在 S 发出时的前 40 字节为图中 S 行，该 IP 分组到达 H 时经过了多少个路由器？

解析。不要先猜“第几次握手”，应按“IP 长度 →TCP 起点 → 控制位 → 序号确认号”顺序解码。所有分组首字节 45 表示 IPv4、IHL=5，即 20 B IP 首部，TCP 从第 21 字节开始。1、3、4 号的源地址 `c0 a8 00 08` 为 192.168.0.8，因此 **H 发送的是 1、3、4 号**；2、5 号的源地址为 S。

编号	IP 总长/B	TCP 首长/B	TCP 标志
1	48	28	SYN
2	48	28	SYN、ACK
3	40	20	ACK
4	56	20	PSH、ACK
5	40	20	ACK

编号	seq（十六进制）	ack（十六进制）	应用数据/B
1	846b41c5	00000000（ACK=0）	0
2	e0599fef	846b41c6	0
3	846b41c6	e0599ff0	0
4	846b41c6	e0599ff0	16
5	e0599ff0	846b41d6	0

1 号的 TCP 数据偏移高半字节为 7，首部长 $7 \times 4 = 28$ B；控制字节 02 置 SYN。2 号的控制字节 12 同时置了 SYN 与 ACK，确认 H 的初始序号加 1；3 号的控制字节 10 只置 ACK，确认 S 的初始序号加 1。因此 **1、2、3 号完成三次握手**，SYN 分别消耗一个序号，第三次纯 ACK

不变量·按 ACK 事件依次更新四账

先更新累计确认，再核对已发未确认区间，随后按拥塞算法改变 cwnd 并读当前 rwnd，最后取小扣在途。若只把两个窗口数字取小就回答还能发多少，会在所有有在途数据的题目中多算。

不额外消耗序号。捕获表只给前 40 B，1、2 号的 8 B TCP 选项没有完整列出，但长度字段足以判定它们没有应用数据。

快速以太网沿用无标签以太网数据字段至少 46 B 的限制，判断填充应比较**整个 IP 分组**与 46 B，不能只比较 TCP 数据。3、5 号 IP 总长 `0028` 为 40 B，均需补 $46-40=6$ B；1、2 号为 48 B、4 号为 56 B，均不需补。因此填充的是 **3、5 号**，每个形成 $14+40+6+4=64$ B 的最短 MAC 帧。填充不计入 IP 总长度，更不占 TCP 字节序号。

第（2）问利用累计确认而不是把 40 字节截取范围当作完整数据。H 首个应用字节的序号为 SYN 序号加 1，即 `846b41c6`；5 号通告下一个期望序号 `846b41d6`，所以 S 已按序收到 `846b41d6` − `846b41c6` = `10` = **16** B 应用数据。这也与 4 号的 $56-20-20=16$ B 相符。

第（3）问先匹配报文身份。S 行 IP 标识为 `6811`，TCP 序号、确认号与 5 号一致，对应 H 处的 5 号；途中的 NAT 可能改变目的地址与端口，不能因此误认为是另一个报文。S 发出时 TTL 为 `40` 即 64，H 处为 `31` 即 49，每经过一个路由器减 1，故经过 $64-49=$ **15** 个路由器。交换机不减 TTL。图中的其余校验和字节按来源保留；题目只截取了部分字节，选项与有效载荷并未给全，不能据此声称已经验证所有报文的完整校验和。

不变量·按 ACK 事件依次更新四账

先更新累计确认，再核对已发未确认区间，随后按拥塞算法改变 cwnd 并读当前 rwnd，最后取小扣在途。若只把两个窗口数字取小就回答还能发多少，会在所有有在途数据的题目中多算。

5.9.4 例 5-9　接收缓存只进不出，窗口最终怎样归零

例 5.9.4 (配套例题)

配套例题（改编）。　下图中 H3 访问 Web 服务器 S。S 为新建 TCP 连接分配 20 KB（K=1024）接收缓存，MSS=1 KB，平均 RTT=200 ms。H3 建立连接时初始序号为 100，持续按 MSS 大小发送数据，拥塞窗口初始阈值 32 KB；S 对收到的每个段确认并通告新接收窗口。连接建立后，S 的 TCP 接收缓存只有数据存入、没有数据取出。图中 R2 的 E0 侧采用 201.1.1.0/24。

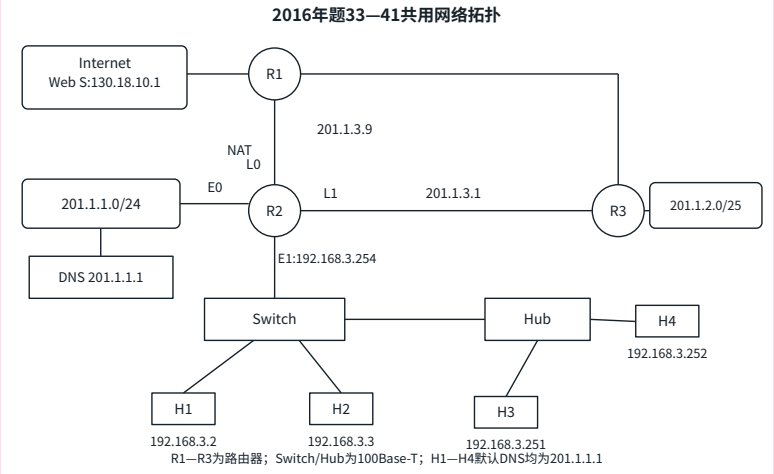


Figure 5.6: H3 为 192.168.3.251，经 Hub 和 R2 访问 130.18.10.1；复用 2016 年共用题图

（1）连接建立时，H3 收到的第二次握手 TCP 段的 SYN、ACK 标志位分别是多少？确认号是多少？

（2）H3 收到的第 8 个确认段所通告的接收窗口是多少？此时拥塞窗口变为多少？发送窗口变为多少？

（3）发送窗口等于 0 时，下一个待发送的数据段序号是多少？

从 H3 发送第 1 个数据段到发送窗口等于 0，平均数据传输速率是多少？忽略段的传输时延。

（4）H3 与 S 通信已结束，H3 在 t 时刻请求断开连接。从 t 起，S 释放该连接的最短时间是多少？

不变量·按 ACK 事件依次更新四账

先更新累计确认，再核对已发未确认区间，随后按拥塞算法改变 cwnd 并读当前 rwnd，最后取小扣在途。若只把两个窗口数字取小就回答还能发多少，会在所有有在途数据的题目中多算。

解析。第（1）问的第二次握手要同时表达 “接受对方 SYN” 和 “发出本方 SYN”，所以 **SYN=1、ACK=1、确认号 =101**。初始序号 100 被 SYN 占用，首个数据字节从 101 开始。

第（2）问必须分别更新两种窗口。沿用教材模型：初始拥塞窗口为 1 MSS，每个新数据段的 ACK 使慢开始窗口增加 1 MSS。第 8 个数据确认到达后，$cwnd = (1+8)\times 1 = \mathbf{9\ KB}$，仍小于 32 KB 阈值；S 已经缓存 8 KB 且未取出，通告 $rwnd = 20 - 8 = \mathbf{12\ KB}$。发送窗口总额度为 $\min(cwnd, rwnd) = \mathbf{9\ KB}$。如果题目问此时能再发送多少，还须扣除已经发出但未确认的数据，不能把 9 KB 直接当成新增发送量。

第（3）问要计到零窗口通告回到 H3 的时刻。每轮受拥塞窗口与剩余接收缓存共同限制，不能把慢开始序列无限写成 1、2、4、8、16。

轮次	发送前 cwnd/MSS	尚余接收缓存/KB	本轮发送/KB
1	1	20	1
2	2	19	2
3	4	17	4
4	8	13	8
5	16	5	5

轮次	累计已接收/KB	本轮确认到达时刻
1	1	1 RTT
2	3	2 RTT
3	7	3 RTT
4	15	4 RTT
5	20	5 RTT

第 5 轮只能再放入 5 KB，S 随后通告零窗口。下一个待发送字节的序号为 $101 + 20\times 1024 = \mathbf{20581}$，这个值不因窗口暂时关闭而重置。计时为 5×200 ms $= 1$ s，平均速率为 $20\times 1024\times 8/1 = \mathbf{163840\ bit/s} = \mathbf{163.84\ kb/s}$。这里 KB 用 1024、kb/s 用 1000；计时起点不包括握手，但终点包括最后一轮确认回传。

零窗口是接收方流量控制的结果，并不说明此前有数据丢失，也不要求按超时规则把 cwnd 重置为 1。发送方可用持续计时器触发零窗口探测，以免窗口更新丢失后永久等待；探测本身不会使接收缓存腾空。在本题 “始终不取数据” 的条件下，不能假设经过一个 RTT 窗口便自动恢复。

第（4）问求 S 的释放时间。为取最短值，S 收到 H3 的 FIN 后立即合并发出 ACK 和自己的 FIN：H3 的 FIN 到 S 用 0.5 RTT，S 的 FIN 到 H3 再用 0.5 RTT，H3 的最后 ACK 返回 S 再用 0.5 RTT。S 收到此 ACK 后进入 CLOSED，故最短为 $1.5\text{RTT} = \mathbf{300\ ms}$。H3 作为主动关闭方的 TIME-WAIT 等待属于另一端的计时，不能再加到 S 的 300 ms 上。

5.10 传输端点、报文边界与内核缓冲

课程连接·两层缓冲分开看

应用缓冲与内核 TCP 收发缓冲不同；对端应用读取速度会影响接收窗口。

同一主机的 TCP 端口空间与 UDP 端口空间相互独立。熟知端口为 0～1023，登记端口为 1024～49151，动态或私有端口为 49152～65535；具体操作系统对临时端口分配和权限的规定应按环境判断。端口字段只有 16 bit，TCP 连接仍可通过四元组区分，同一服务端口可同时对应多条连接。

应用缓冲区与 TCP 发送、接收缓冲区属于不同层次。应用把数据交给发送缓冲区之后，传输层根据窗口、MSS 和协议状态安排分段；已发送而未确认的字节还要保留以便重传。接收方可以缓存失序字节，但按顺序交给应用。应用尚未读取时，已收到的数据继续占用接收缓冲；应用读取并释放空间后，窗口才可相应扩大。

应用先写 1600 B、再写 1400 B，MSS=1000 B 时，TCP 可以划成若干不超过 1000 B 的段，第二段也可能跨越两次应用写入边界；接收应用应依自身报文格式识别边界。MSS 表示单段数据上限，并不要求每段必须装满。普通 IPv4 首部 20 B、TCP 首部 20 B、路径 MTU1500 B 时，简单模型 MSS 可取 1460 B；选项和路径条件改变后须重新核对。

UDP 则保留应用报文边界。课件中源端口 3666、目的端口 9888、应用报文 96 B，对应 UDP 总长度 104 B；若 IPv4 首部 20 B，IP 总长度 124 B。UDP 应用报文与 UDP 总长度、IP 总长度是三层边界，分片时也不能擅自给每片重复增加一套 UDP 首部。

5.11 反码检验的发送端、接收端与漏检

取课件的两个 16 bit 字 `0x5065`、`0x6951`，和为 `0xB9B6`，按位取反得到检验和 `0x4649`。再加入第三个字 `0x65C5`，普通和为 `0x11F7B`，回卷得到 `0x1F7C`，检验和为 `0xE083`。接收方把数据字和检验和一起求反码和，应得到 `0xFFFF`。

通过校验仅表示未检出差错。例如两个字分别增加和减少同一个数，某些情况下会保持相同反码和，因而漏检。协议允许关闭校验的条件也要区分：IPv4 UDP 字段全 0 可表示未使用校验；若实际计算值为 0，应按协议写成全 1，避免与“未校验”混淆。IPv6 的普通 UDP 使用条件不能照搬 IPv4 的可选规则。

5.12 逐字节维护确认号与接收窗口

设服务器接收缓冲为 8 B，应用暂不读取；客户下一数据序号为 600，服务器本方向下一序号为 200。客户先发送 600～602 共 3 B，服务器累计确认号 603，剩余窗口 5 B；再收到 603～604 与 605～606 两个 2 B 段后，累计确认 607，剩余窗口 1 B；收到 607 这一字节后缓冲满，累计确认 608、窗口 0。若应用随后读取全部 8 B，可通告窗口 8 B，确认号仍为 608。

新收到的客户数据	连续已收到至	ACK	应用未读时 rwnd/B
600～602	602	603	5
603～604	604	605	3
605～606	606	607	1
607	607	608	0
应用读取 8 B，无新数据	607	608	8

若接收方采用累计确认，连续两个 2 B 段也可以只发最终 ACK607；表中逐段列出是为了说明各次状态。服务器反方向发送 5 B 数据时序号从 200 开始，客户返回 ACK205；这个 205 只确认服务器字节流，与客户已经发送到 607 的位置相互独立。捎带确认把反方向的新数据与本方向接收进度放进同一段，两个序号空间仍须分开记账。

计数·原 ACK 加三个重复

首次 ACK623 推进确认；再收到三个相同的重复 ACK 才达经典快重传阈值。

5.13 快重传必须逐次计数

客户连续发送序号 622、623、624、625 的四个 1 B 段，623 丢失；假设接收方在 622 之前已经连续收到至 621，则收到 622 后首次返回 ACK623。收到 624 和 625 后各重复 ACK623，这时只有 **2 个重复 ACK**，尚未达到 3 个重复 ACK 阈值。若随后 626 也到达并再次返回 ACK623，才得到第 3 个重复 ACK，触发重传缺失的 623。

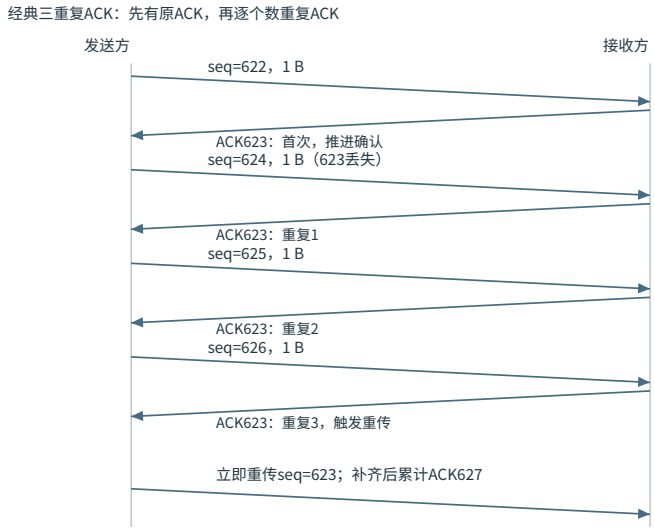


Figure 5.7: 第一个 ACK623 推进确认，后三个相同确认分别计重复 1、2、3

若总共只有 622～625 四段而没有更多到达事件，经典三重复 ACK 规则下应等超时等机制；不能为了让图符合预想结果，把首次正常 ACK 也算入重复次数。收到重传 623 且 624～626 已经缓存时，累计确认可直接推进到 627，无须让发送方重新发送已缓存的数据。

教材按轮次讨论拥塞窗口时，若门限 16 MSS，慢开始发送轮次为 1、2、4、8、16，随后拥塞避免为 17、18……。在 cwnd=24 MSS 时超时，门限取 12 MSS、窗口重置 1 MSS，再经 1、2、4、8、12 后转为 13、14……；在相同点触发快重传并完成课件简化快恢复，则从 12 MSS 进入拥塞避免。窗口增长轨迹必须同时写清初值、门限、丢包事件和记录时刻。

5.14 主动回忆自测

以下为自拟题。每道窗口题都画出本方向字节区间，并说明采用的教材算法。

1. A 发 SYN 时 seq=100，之后发送 500 B 应用数据，再发不带数据的 FIN。FIN 的 seq、对方确认 FIN 的 ack 分别是多少？

2. 最新有效 ack=5001、rwnd=4000 B、cwnd=3000 B；已发未确认区间 5001～6500。还可新发多少？最大新字节序号是多少？
3. 被动关闭端进入 CLOSE-WAIT 后，是否已经不能再发应用数据？
4. 一份最初的正常 ACK 加两份相同的重复 ACK，是否达到 “三个重复 ACK” 的计数？
5. TCP 能否像 UDP 一样把两次应用写入的边界自动交给接收应用？服务器同一个 80 端口能否同时服务多个 TCP 客户？
6. 12 B 应用数据装入 UDP，总长度字段是多少？伪首部和求和补齐的 0 字节会在线上成为额外 UDP 数据吗？
7. IP 总长 100 B、IHL=6、TCP 数据偏移 =6，未分片，应用数据多少字节？
8. 超时重传后收到一个 ACK，为什么不能总用它的到达时刻减第一次发送时刻当 RTT 样本？
9. rwnd 降至 0 是否证明网络拥塞？用于防止零窗口通知丢失后永久等待的计时器是什么？
10. 旧 SRTT 为 100 ms、平滑系数 1/8、新有效样本 140 ms，新 SRTT 是多少？Reno 与 Tahoe 遇到三个重复 ACK 后的行为是否必相同？

5.14.1 核对答案

1. 数据 101～600，FIN seq=601，对方 ack=602。
2. 总额度 3000 B，已占 1500 B，尚可发 1500 B；允许新发 6501～8000。
3. 尚可发送，直到本机应用关闭发送方向；收到 FIN 只关闭对端到本端方向。
4. 没有，首次正常 ACK 之后还需 3 份重复 ACK。
5. 不保证，TCP 是字节流；可以，源 IP/端口等使四元组不同。
6. 20 B；两者仅用于校验计算，均不会变成额外应用载荷。
7. $100 - 4 \times 6 - 4 \times 6 = 52$ B。
8. ACK 可能确认原段也可能确认重传段，样本有歧义；Karn 原则应排除相应有歧义样本。
9. 不证明，可能只是接收缓存不足；使用持续计时器触发零窗口探测。
10. 105 ms；不相同，Reno 快恢复与 Tahoe 退回慢开始的处理不同，按具体题设算法判断。

计数·原 ACK 加三个重复

首次 ACK623 推进确认；再收到三个相同的重复 ACK 才达经典快重传阈值。